Learning LangGraph: The 20% That Matters

1 Why LangGraph Exists

You can build an LLM agent with a plain `while` loop: call the model, check if it wants a tool, run the tool, loop. That works for demos but breaks in production — there is no persistence, no way to pause for human approval, and no inspectability (the logic is buried in if/else chains). LangGraph fixes this by making the loop an **explicit directed graph**: nodes for computation, edges for control flow, and a shared state object that flows through them. It reached stable v1.0 in October 2025.

Use LangGraph when your LLM app is more than a single call: it loops, branches, calls tools, needs memory across turns, coordinates multiple agents, or pauses for a human.

2 The Three Core Concepts

- **State** — a shared object (a `TypedDict`) that every node reads from and writes to. It is the single source of truth flowing through the graph.
- **Nodes** — plain Python functions. Each takes the state, does work (call an LLM, run a tool), and returns a partial update to the state.
- **Edges** — connect nodes, defining what runs next. Edges can be fixed or **conditional** (the path depends on the state).

You define these, then `compile()` the graph into something runnable.

3 Setup

```
pip install langgraph langchain langchain-openai
```

Deprecation warning: many older tutorials use `set_entry_point()` and `set_finish_point()`. These are deprecated. The current API uses the special `START` and `END` nodes with `add_edge`. If you hit import errors following a guide, it is probably pre-v1.0.

4 Step 1 — Define the State

The state is a `TypedDict`. For fields that should *accumulate* (like a message history) you attach a **reducer** so updates append instead of overwrite.

```
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
```

```
    # add_messages reducer: new messages are APPENDED, not replaced
    messages: Annotated[list, add_messages]
```

Key idea — reducers: without a reducer, returning `{"messages": [m]}` from a node would overwrite the list. The `add_messages` reducer makes it append, which is what you want for a conversation. For chat agents, this messages-with-reducer pattern is so common that LangGraph ships a prebuilt `MessagesState` you can subclass.

5 Step 2 — Write Nodes

A node is a function: state in, partial state update out.

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini")
llm_with_tools = llm.bind_tools(tools)   # tells the LLM which tools exist

def call_llm(state: AgentState):
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}      # appended via the reducer
```

6 Step 3 — Build and Wire the Graph

This is the canonical ReAct agent loop — the single most important pattern to memorize.

```
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode

graph = StateGraph(AgentState)

# Add nodes (name, function)
graph.add_node("llm", call_llm)
graph.add_node("tools", ToolNode(tools))   # prebuilt node that runs tool
    calls

# Wire edges
graph.add_edge(START, "llm")               # entry point
graph.add_conditional_edges("llm", should_continue, {
    "tools": "tools",                      # if a tool was called -> run
        it
    END: END,                              # otherwise -> finish
})
graph.add_edge("tools", "llm")             # after tools, go back to the
    LLM

app = graph.compile()
```

The shape of this graph:

```
START -> llm --(tool call?)--> tools -> llm
              \--(no)--------> END
```

7 Step 4 — The Conditional Edge Function

A conditional edge is a function returning a string (the key in the routing dict).

```python
def should_continue(state: AgentState) -> str:
    last = state["messages"][-1]
    if last.tool_calls:          # the LLM requested a tool
        return "tools"
    return END                   # the LLM gave a final answer
```

This single function is how the agent "decides" whether to keep working or stop. Most of your custom logic lives in conditional edges like this.

8 Step 5 — Run It

```python
# One-shot result
result = app.invoke({"messages": [{"role": "user", "content": "What's 2+2
    times 10?"}]})
print(result["messages"][-1].content)

# Stream node-by-node to watch it reason
for update in app.stream(initial_state, stream_mode="updates"):
    for node_name, node_update in update.items():
        print(node_name, "->", node_update["messages"][-1])
```

`invoke` gives the final state; `stream` emits one event per node execution, which is ideal for debugging and for showing progress in a UI.

9 Tools

A tool is just a function the LLM can call. Decorate it and bind it.

```python
from langchain_core.tools import tool

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""    # docstring matters: the LLM reads it
    return a * b

tools = [multiply]
```

`ToolNode(tools)` (used above) automatically executes whatever tool the LLM requested and returns the result into the state.

10 The Fast Path: `create_react_agent`

For a standard tool-calling agent you do not need to build the graph by hand. LangGraph ships a prebuilt that constructs exactly the loop above:

```python
from langgraph.prebuilt import create_react_agent

agent = create_react_agent(llm, tools=tools)
result = agent.invoke({"messages": [{"role": "user", "content": "..."}]})
```

When to use which: start with `create_react_agent`. Drop down to a hand-built `StateGraph` when you need custom nodes, parallel branches, or human-in-the-loop interrupts.

11 Memory and Persistence (Checkpointing)

By default the graph is stateless — it forgets everything after each `invoke`. A **checkpointer** saves the state after every node, giving you multi-turn memory, fault tolerance, and the ability to pause and resume.

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
app = graph.compile(checkpointer=checkpointer)

# A thread_id identifies a conversation/session
config = {"configurable": {"thread_id": "user-42"}}

app.invoke({"messages": [{"role": "user", "content": "Hi, I'm Sam"}]},
    config)
app.invoke({"messages": [{"role": "user", "content": "What's my name?"}]},
     config)
# -> remembers "Sam" because state is checkpointed under thread_id "user
    -42"
```

Key idea: the `thread_id` is the session key. Same id = same continuing conversation. Use `InMemorySaver` for development; swap in a database-backed saver for production.

12 Human-in-the-Loop (Awareness)

Because the checkpointer can snapshot and resume, you can make the graph **pause** before a sensitive step (e.g. sending an email), wait for a human to approve, then continue exactly where it left off. This "interrupt" capability is a major reason to use LangGraph over a raw loop. You add it once you are comfortable with the basics.

13 Multi-Agent (Awareness)

You can put a whole agent inside a node. A common pattern is a **supervisor**: one node routes work to specialized agents (researcher, writer, reviewer), each itself a `create_react_agent`, via a conditional edge that reads a "next" field in the state. Same primitives — nodes, edges, state — just nested.

14 The Mental Model in One Line

LangGraph is a state machine for LLMs: a shared **state** flows through **nodes** (functions) along **edges** (control flow), with **conditional edges** for branching and a **checkpointer** for memory and pause/resume.

15 The Checklist for Any Graph

1. Define the **State** (TypedDict; add reducers for accumulating fields).

2. Write **nodes** as functions returning partial state updates.
3. Add edges: `START` → first node, **conditional edges** for branching, `END` to finish.
4. `compile()` (add a **checkpointer** if you need memory).
5. Run with `invoke` (final) or `stream` (step-by-step).
6. Pass a `thread_id` to keep conversations separate.

16 Practice Task

1. Define an `AgentState` with a reduced `messages` field.
2. Write two tools (e.g. `multiply` and a fake `get_weather`).
3. Build the ReAct graph by hand: `llm` node, `ToolNode`, conditional edge, loop back.
4. Run it with `stream` and watch it call a tool then answer.
5. Rebuild the same thing in one line with `create_react_agent` and confirm they behave alike.
6. Add an `InMemorySaver`; in one thread tell it your name, then in a second message ask it to recall your name.
7. Bonus: wrap the agent in a FastAPI `POST /chat` endpoint (passing `thread_id`), and give it a tool that runs your RAG `rag()` function from the previous document.